

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Short Answer Text(High) Assessment

Tool Weightage: 40%

QUESTIONS		
Q. No	Question	Bloom's Level
1	Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema.	BL1 – Remember
2	Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram.	BL2 – Understand
3	Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation.	BL2 – Understand
4	Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities.	BL2 – Understand

5	Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position.	BL1 – Remember
---	---	----------------

Code of Conduct:

I _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	30%	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Linkages	25%	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Organization	20%	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Integration of Literature	15%	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity	10%	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1.Schema

Definition

A **schema** is the overall structure or design of a database. It defines how data is organized, including tables, attributes, relationships, and constraints. An **instance** is the actual data stored in the database at a particular point in time.

Example

Consider a **Student** table:

Student Table Schema

Attributes	Datatypes
Student_ID	Integer
Name	Varchar(50)
Department	Varchar(50)
Age	Integer

The schema can be represented as:

Student(Student_ID, Name, Department, Age)

This defines the structure of the Student table.

2. Instance

Definition

An instance is the collection of data stored in the database at a specific moment. It changes whenever data is added, updated, or deleted.

Example

Student_ID	Name	Department	Age
101	Gunavathi	CSE	22
102	Vicky	ECE	21
103	Poornima	IT	20

This table represents one instance of the Student database.

1. Logical Schema

Definition

The logical schema defines the logical structure of the database. It includes tables, attributes, relationships, primary keys, and foreign keys, but it does not describe how the data is physically stored.

Example:

- Student (Student_ID, Name, Department, Age)
- Course (Course_ID, Course_Name, Credits)

Features:

- Defines tables and attributes.
- Specifies relationships between tables.
- Independent of physical storage.
- Used by database designers.
- .

2. Physical Schema

Definition

The physical schema describes how the database is physically stored on storage devices. It includes file organization, indexes, storage locations, and access methods.

Example:

- Student records are stored in a file named **student.dat**.
- An index is created on **Student_ID** for faster searching.
- Data is stored in disk blocks.

Features:

- Specifies storage details.
- Improves database performance.
- Managed by the DBMS and Database Administrator (DBA).

3. View Schema (External Schema)

Definition

A view schema defines how different users see the database. It displays only the required information and hides unnecessary or confidential data.

Example:

Original Student Table

Student_ID	Name	Department	Age	Phone
101	Gunavathi	CSE	22	9876543210

Faculty View

Student_ID	Name	Department
101	Gunavathi	CSE

Here, the **Age** and **Phone** columns are hidden.

Features:

- Provides security.
- Hides confidential information.
- Different users can have different views.

Difference Between Logical Schema, Physical Schema, and View Schema

Feature	Logical schema	Physical Schema	View Schema
Meaning	Describes the logical structure of the database.	Describes how data is physically stored.	Describes how users view the database.
Focus	Tables, attributes, relationships, keys, and constraints.	Storage files, indexes, disk organization, and access methods.	User-specific data presentation.
Storage Details	Not included.	Included	Not included
Used By	Database designers and developers.	Database administrators (DBAs).	End users and applications.
Purpose	Organize data logically.	Improve storage efficiency and performance.	Provide security and simplify data access.
Example	Student(Student_ID, Name, Department, Age)	Data stored in student.dat with an index on Student_ID.	Faculty view showing only Student_ID, Name, and Department.

Conclusion

A **schema** defines the structure of a database, while an **instance** represents the actual data stored in the database at a particular time. The **logical schema** describes the database structure, the **physical schema** explains how the data is stored, and the **view schema** provides customized views of the database for different users. These schemas help in designing, storing, and accessing data efficiently and securely.

2. Bank Management System – ER Diagram

Aim

To identify the **entities, attributes, and relationships** in a **Bank Management System** and construct an **Entity Relationship (ER) Diagram**.

1. Entities and Attributes

Entity 1: Customer

Attributes:

- Customer_ID (Primary Key)
- Customer_Name
- Address
- Phone_No
- Email

Entity 2: Account

Attributes:

- Account_No (Primary Key)
- Account_Type
- Balance
- Open_Date

Entity 3: Branch

Attributes:

- Branch_ID (Primary Key)
- Branch_Name
- Branch_Address
- IFSC_Code

Entity 4: Loan**Attributes:**

- Loan_ID (Primary Key)
- Loan_Type
- Loan_Amount
- Interest_Rate

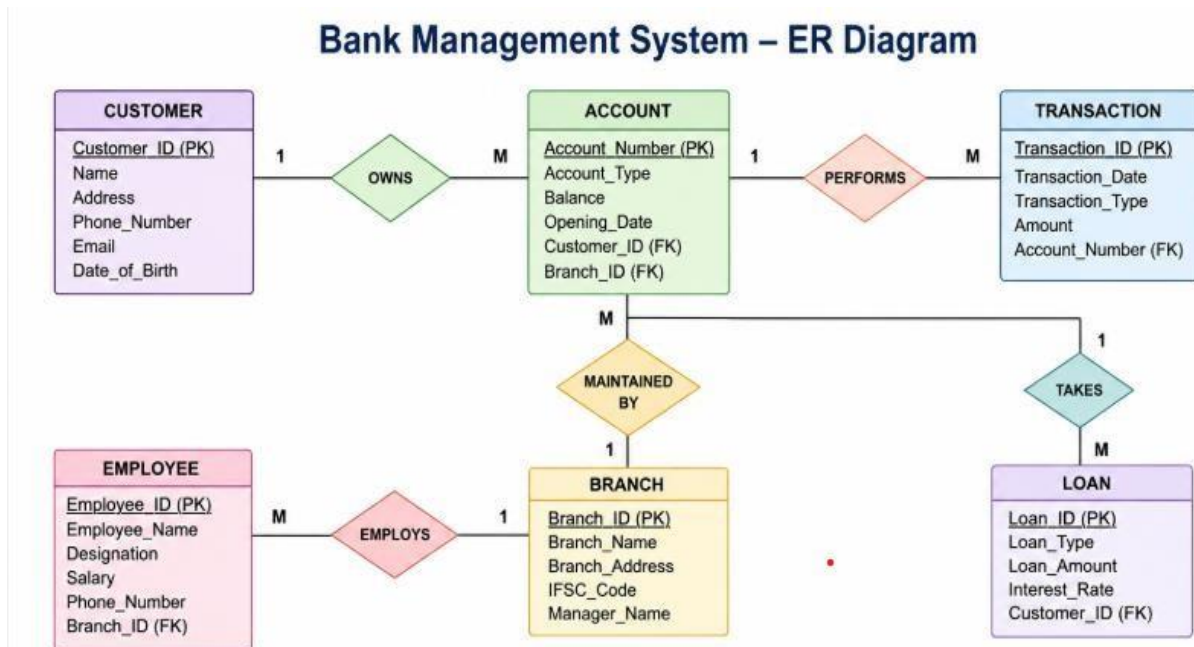
Entity 5: Employee**Attributes:**

- Employee_ID (Primary Key)
- Employee_Name
- Designation
- Salary

2. Relationships

Relationship	Description	Cardinality
Customer-Account	A customer owns one or more accounts.	One-to-Many (1:M)
Branch-Account	A branch maintains many accounts.	One-to-Many (1:M)
Customer-Loan	A customer can take one or more loans.	One-to-Many (1:M)
Branch-Employee	A branch employs many employees.	One-to-Many (1:M)
Branch-Loan	A branch sanctions many loans.	One-to-Many (1:M)

3. ER Diagram



Conclusion

A Bank Management System consists of five main entities: Customer, Account, Branch, Loan, and Employee. These entities are connected through relationships such as owns, maintains, employs, sanctions, and takes. The ER diagram helps represent the database structure clearly and is useful for designing an efficient bank database.

3. Extended ER (EER) Model

Definition

The **Extended Entity Relationship (EER) Model** is an advanced version of the ER model that includes additional concepts to represent complex real-world entities and relationships. It extends the basic ER model by adding concepts such as **generalization, specialization, and aggregation**.

The EER model is mainly used for designing large and complex databases.

1. Generalization

Definition

Generalization is a bottom-up approach in which two or more lower-level entities are combined to form a higher-level entity based on their common attributes.

It creates a **superclass** from multiple subclasses.

Example

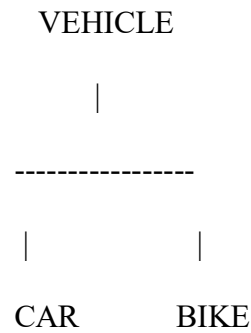
Entities:

- Car
- Bike

Both have common attributes:

- Vehicle_ID
- Color
- Model

They can be generalized into a higher-level entity:

**Explanation:**

Car and Bike are specialized types of Vehicle. The common properties are stored in the Vehicle entity.

Features

- Combines similar entities into one higher-level entity.
- Reduces data redundancy.
- Follows a bottom-up approach.

2. Specialization**Definition**

Specialization is a top-down approach in which a higher-level entity is divided into two or more lower-level entities based on their specific characteristics.

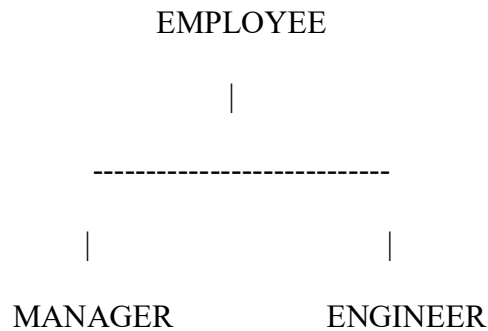
It creates **subclasses** from a superclass.

Example

Entity:

- Employee

Employee can be divided into:

**Explanation:**

Manager and Engineer have their own specific attributes along with the common Employee attributes.

Features

- Divides one entity into multiple specialized entities.
- Follows a top-down approach.
- Subclasses inherit attributes from the superclass.
- Helps represent different roles clearly.

3. Aggregation**Definition**

Aggregation is a concept in EER modeling where a relationship between entities is treated as a higher-level entity. It is used when a relationship itself needs to participate in another relationship.

Example

Consider:

- Employee works on a Project.
- A Manager monitors this work relationship.

EMPLOYEE ----- Works_On ----- PROJECT

|
(Aggregation)

|
MONITOR

|
MANAGER

Explanation:

The **Works_On** relationship is treated as an entity so that Manager can have a relationship with it.

Features

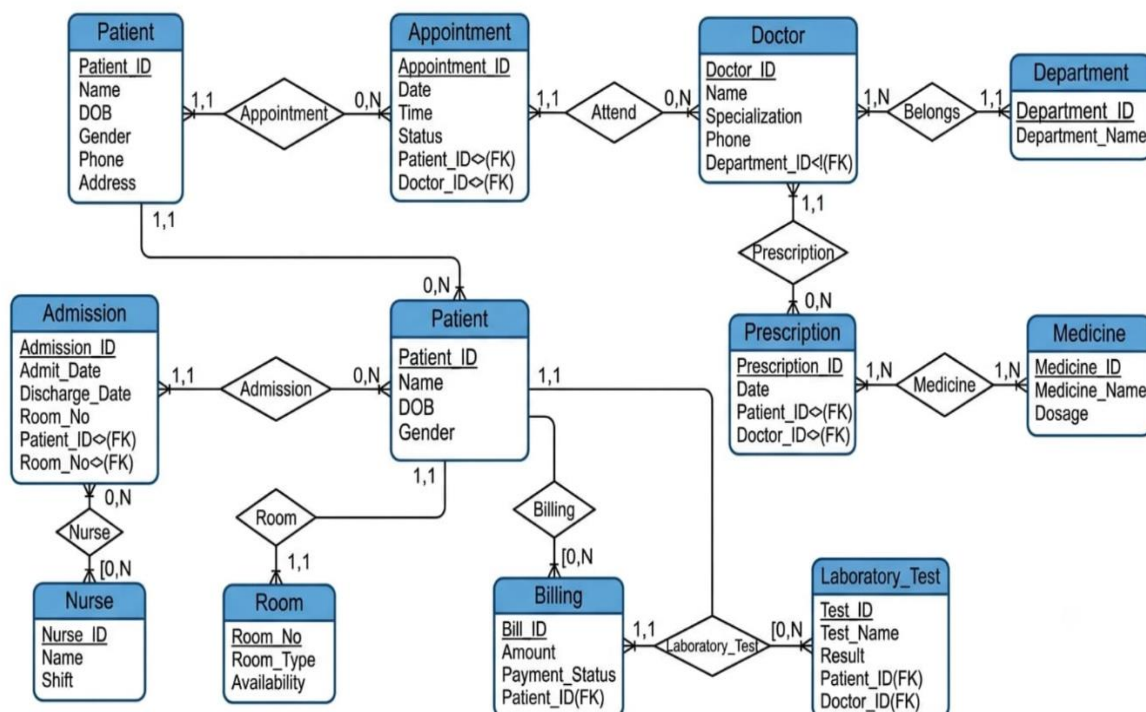
- Represents relationships between relationships.
- Used for complex relationship modeling.
- Helps simplify complex ER diagrams.

Advantages of the EER Model

- Represents complex real-world situations.
- Supports inheritance of attributes.
- Reduces redundancy.
- Improves database organization.
- Makes database design more flexible and efficient.

Conclusion

The **EER model** enhances the traditional ER model by providing additional features for representing complex database structures. **Generalization** combines similar entities into a common superclass, **specialization** divides an entity into subclasses, and **aggregation** represents relationships as higher-level entities. These concepts help in designing more accurate and efficient database models.



5. Role of a DBA (Database Administrator)

Definition

A **Database Administrator (DBA)** is a person responsible for managing, maintaining, and controlling the database system. The DBA ensures that the database is secure, available, reliable, and performs efficiently.

The DBA acts as a link between the users and the database system and manages all database-related activities.

Responsibilities of a DBA

1. Database Design and Creation

- Designs the database structure according to the requirements.
- Creates tables, relationships, constraints, and schemas.
- Ensures proper organization of data.

2. Database Installation and Configuration

- Installs and configures database management systems (DBMS).
- Sets up database servers and required software.
- Updates database software when needed.

3. Security Management

- Controls user access to the database.
- Creates user accounts and assigns permissions.
- Protects confidential data from unauthorized access.

4. Backup and Recovery

- Performs regular database backups.
- Restores data during system failures or data loss.
- Develops recovery plans to prevent data loss.

5. Performance Monitoring and Optimization

- Monitors database performance.
- Improves query execution speed.
- Creates and manages indexes for faster data retrieval.

6. Data Integrity Management

- Ensures accuracy and consistency of data.
- Maintains constraints such as primary keys and foreign keys.
- Prevents invalid data entry.

7. User Management

- Creates and manages database users.
- Provides different levels of access based on user requirements.
- Handles user-related database issues.

8. Database Maintenance

- Performs regular database updates and maintenance tasks.
- Checks for errors and resolves database problems.
- Ensures continuous availability of the database.

9. Transaction Management

- Ensures proper execution of database transactions.
- Maintains properties of transactions like consistency and reliability.
- Handles concurrency issues when multiple users access data simultaneously.

Skills Required for a DBA

- Knowledge of DBMS concepts.
- Understanding of SQL and database languages.
- Knowledge of database security and backup methods.
- Problem-solving and troubleshooting skills.

Conclusion

A **Database Administrator (DBA)** plays an important role in managing a database system. The DBA is responsible for database design, security, backup, recovery, performance improvement, and maintenance. Effective DBA management ensures that databases remain secure, reliable, and available to users.

.